

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

**Verilog Assignment: FPGA-Based Signed
Multiply–Accumulate Unit**

Using the Previously Designed Fast Booth Multiplier

Instructions

This is a group assignment. Each group must design, simulate, synthesize, implement, and demonstrate the complete design on the FPGA board. The final board demonstration is mandatory.

The purpose of this assignment is to reuse the fast signed Booth multiplier developed in the previous assignment and build a simple sequential **Multiply–Accumulate (MAC)** system around it.

The design will use fixed operand values supplied through the slide switches. After reset is released, the accumulator will repeatedly add the same multiplication result at a slow, visible rate. The current 16-bit accumulator value will be displayed directly on the 16 user LEDs.

1. Objective

Design a signed MAC unit that repeatedly performs

$$ACC_{new} = ACC_{old} + A \times B$$

where:

- A is an 8-bit signed two's-complement operand;
- B is an 8-bit signed two's-complement operand;
- ACC is a 16-bit accumulator;
- A and B are supplied using the 16 slide switches on the FPGA board; and
- the current accumulator value is displayed using the 16 LEDs.

The multiplication must be performed using the **Fast Booth multiplier developed in the previous assignment**. The Verilog multiplication operator `*` must not be used in the MAC design.

2. Basic Operation

The two operands remain fixed while the MAC is running. Students should first set the switch values for A and B , assert reset, and then release reset.

Immediately after reset,

$$ACC = 0.$$

At every visible MAC update,

$$ACC \leftarrow ACC + (A \times B).$$

Therefore, if the multiplication result is

$$P = A \times B,$$

then the accumulator sequence is

$$0, P, 2P, 3P, 4P, \dots$$

and the LEDs must change accordingly.

3. FPGA Board Interface

Use the 16 slide switches as two signed 8-bit operands.

Board Resource	Signal	Function
100 MHz clock	clk	System clock
Pushbutton	reset	Active-high reset
SW[7:0]	A[7:0]	Signed 8-bit operand A
SW[15:8]	B[7:0]	Signed 8-bit operand B
LED[15:0]	led[15:0]	Current 16-bit accumulator value

Thus, the switch organization is

$$SW[15:8] = B[7:0] \qquad SW[7:0] = A[7:0].$$

The most-significant bit of each operand is the sign bit:

$$A[7] = \text{sign bit of } A,$$

$$B[7] = \text{sign bit of } B.$$

For the mandatory board demonstration, the operands should remain fixed after reset is released.

4. Reuse of the Previous Fast Booth Multiplier

The previous assignment implemented a signed 32-bit Fast Booth multiplier using radix-4 Booth recoding and Carry-Save reduction. That multiplication hardware must be reused in this assignment.

Since the board provides only 8 bits for each operand, first sign-extend each input to 32 bits:

$$A_{32} = \{\{24\{A[7]\}\}, A[7:0]\},$$

$$B_{32} = \{\{24\{B[7]\}\}, B[7:0]\}.$$

These values are supplied to the Fast Booth multiplier.

The multiplier produces a signed 64-bit result:

$$P_{64} = A_{32} \times B_{32}.$$

Because an 8-bit signed multiplication always fits within 16 bits, use

$$P_{16} = P_{64}[15:0]$$

as the value added to the accumulator.

Students are **not required to redesign the Booth encoder, partial-product generator, or CSA tree**. They should reuse the multiplication datapath/core developed in the previous assignment.

If the previous design contains the multiplier datapath inside a larger **start/busy/done** wrapper, students may expose and reuse the multiplier arithmetic datapath as a separate submodule for this assignment.

5. Slow Visible MAC Update

The FPGA clock is too fast for the LED changes to be visible directly. Therefore, create a clock divider similar to the slow-clock circuit used earlier in the laboratory.

For example, a counter may be driven by the 100 MHz board clock:

```
reg [25:0] clk_div;

always @(posedge clk or posedge reset) begin
    if (reset)
        clk_div <= 26'd0;
    else
        clk_div <= clk_div + 1'b1;
end

wire slow_clk;
assign slow_clk = clk_div[25];
```

Students may select a nearby divider bit if a slightly faster or slower visible update rate is preferred.

The accumulator must update on the rising edge of the slow clock:

```
always @(posedge slow_clk or posedge reset) begin
    if (reset)
        accumulator <= 16'd0;
    else
        accumulator <= accumulator + product_16;
end
```

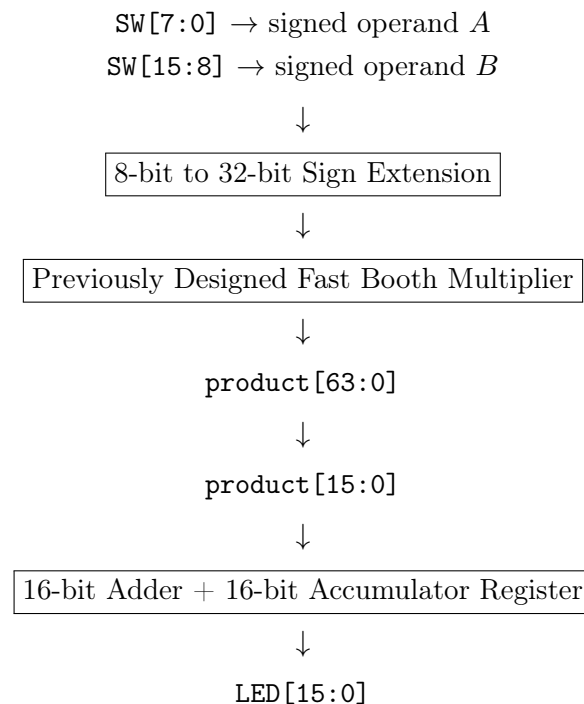
The LEDs must continuously display the accumulator:

```
assign led = accumulator;
```

The above code is illustrative. Students must correctly connect the multiplier output and ensure all signed widths are handled properly.

6. Required Datapath

The required data flow is:



The accumulator contains the feedback path required for the MAC operation:

$$ACC \rightarrow \text{Adder} \rightarrow ACC.$$

A separate complex FSM is not required for this assignment. The sequential behavior is provided by the accumulator register and the slow update clock.

7. Top-Level Module

Create a top-level module named

`mac_booth_board`

with an interface equivalent to:

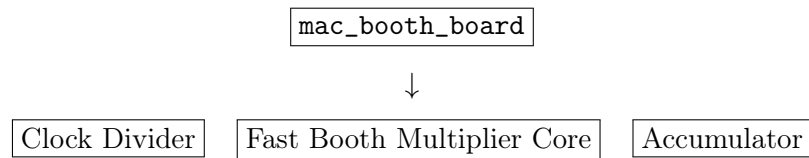
```
module mac_booth_board (
    input wire clk,
    input wire reset,
    input wire [15:0] sw,
    output wire [15:0] led
);
```

Internally interpret

```
A = sw[7:0];
B = sw[15:8];
```

as signed two's-complement numbers.

A suggested module hierarchy is



8. Worked Board Examples

Example 1: Positive $\times$ Positive

Set

$$A = 3 = 00000011,$$

$$B = 2 = 00000010.$$

Therefore,

$$A \times B = 6.$$

After reset, successive visible MAC updates should produce:

MAC Update	Accumulator	LED[15:0]
Reset	0	0000 0000 0000 0000
1	6	0000 0000 0000 0110
2	12	0000 0000 0000 1100
3	18	0000 0000 0001 0010
4	24	0000 0000 0001 1000
5	30	0000 0000 0001 1110

Thus the LED pattern must visibly change on every slow MAC update.

Example 2: Negative $\times$ Positive

Set

$$A = -2 = 11111110,$$

$$B = 5 = 00000101.$$

Therefore,

$$A \times B = -10.$$

The first few accumulator values are

$$0, -10, -20, -30, -40, \dots$$

and the corresponding 16-bit two's-complement LED values are

MAC Update	Accumulator	LED Hex Value
Reset	0	0000
1	-10	FFF6
2	-20	FFEC
3	-30	FFE2
4	-40	FFD8

9. Accumulator Overflow

The accumulator is intentionally limited to 16 bits because only 16 LEDs are available.

Therefore, after enough MAC operations, the mathematical result may exceed the signed 16-bit range

$$-32768 \leq ACC \leq 32767.$$

No saturation or overflow correction is required.

The hardware should simply retain the lower 16 bits of the arithmetic result. Therefore, the accumulator naturally wraps around modulo

$$2^{16}.$$

This behavior is acceptable for this assignment.

During the board demonstration, students must first show several MAC updates for operand values where the correct arithmetic progression can be verified easily before overflow occurs.

10. Behavioral Simulation

Before programming the FPGA, write a testbench for the top-level MAC design or for a suitable simulation version of the design.

The simulation must verify at least:

- positive $\times$ positive accumulation;
- positive $\times$ negative accumulation;
- negative $\times$ negative accumulation;
- reset behavior; and
- repeated accumulation for at least five update cycles.

The waveform should show at least:

`clk, reset, A, B, product_16, slow_clk, accumulator.`

For simulation, students may reduce the divider width so that the slow-clock transitions occur quickly enough to observe multiple MAC operations without an unnecessarily long simulation.

11. FPGA Board Demonstration

Board demonstration to a Teaching Assistant is mandatory.

The required demonstration procedure is:

1. Set a fixed signed value of A using `SW[7:0]`.
2. Set a fixed signed value of B using `SW[15:8]`.
3. Assert reset and verify that all 16 LEDs become zero.
4. Release reset.
5. Observe the LEDs changing at each slow MAC update.
6. Demonstrate that the displayed sequence corresponds to

$$0, A \times B, 2(A \times B), 3(A \times B), \dots$$

for the first few updates.

At minimum, demonstrate:

- one positive $\times$ positive case; and
- one case containing a negative operand.

The TA may provide another small pair of fixed signed operands and ask the group to predict the first few LED values before running the design.

12. Design Restrictions

1. The Verilog multiplication operator `*` must not be used in the design.
2. The Fast Booth multiplication hardware from the previous assignment must be reused.
3. The 8-bit switch operands must be treated as signed two's-complement values.
4. Correct sign extension from 8 bits to 32 bits is required.
5. The MAC accumulator must be exactly 16 bits.
6. Overflow beyond 16 bits may be discarded; wraparound is acceptable.
7. The LED output must show the raw 16-bit accumulator value.
8. The accumulator must update slowly enough that successive values can be observed visually on the board.
9. The accumulator must return to zero whenever reset is asserted.
10. Hard-coding the expected LED sequence for particular operand values is not allowed.

13. Submission

Submit a single zipped folder named

`MAC_Booth_{Group_Number}.zip`.

The submission must contain:

- all Verilog source files for the MAC design;
- the reused Fast Booth multiplier source files required by the project;
- the testbench;
- the FPGA XDC constraints file;
- at least one behavioral simulation waveform screenshot; and
- a simple block diagram showing the multiplier, accumulator feedback path, clock divider, switches, and LEDs.

Each Verilog source file must contain the group number, names and roll numbers of all group members, assignment name, module name, and date of submission.